2025 IEEE 36th International Conference on Application-specific Systems, Architectures and Processors (ASAP) | 979-8-3315-9552-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/ASAP65064.2025.00018

Labidus: RISC-V Overlay with Streaming Asynchronous Custom Instructions

Gongjin Sun, Seongyoung Kang, Jane He, Se-Min Lim, Sang-Woo Jun
Department of Computer Science
University of California, Irvine
Irvine, USA
{gongjins, seongyk3, siyaoh4}@uci.edu, {seminl1, swjun}@ics.uci.edu

***Abstract*—High development complexity is one of the most critical issues preventing more widespread use of reconfigurable hardware accelerators such as FPGAs. While soft processor overlays allow productive development with high-level software tools, they suffer from low performance. We address this issue with Labidus, a parallel RISC-V soft processor overlay that addresses this performance gap while maintaining development simplicity. Labidus automatically generates custom instructions based on static analysis of user software. These custom instructions achieve high utilization through two key innovations: asynchronous semantics and sharing across four-core tiles. Our evaluation across four scientific computing applications shows that Labidus matches or exceeds the performance of even manually optimized FPGA accelerators for gigabyte-scale tasks, at a fraction of development effort.**

***Index Terms*—FPGA, Overlay, RISC-V**

I. Introduction

Field Programmable Gate Arrays (FPGAs) have emerged as a promising technology for application-specific hardware acceleration, offering potential for continued performance and power efficiency scaling in computer systems. Typically deployed as PCIe-attached accelerator cards similar to GPUs, FPGAs have regularly demonstrated superior power efficiency and competitive performance compared to GPUs on many important applications [1], [2]. Unlike fixed-function Application-Specific Integrated Circuits (ASICs), FPGAs can be reconfigured after deployment if the application or algorithm changes [3], [4].

However, these benefits come with a cost of high hardware design complexity and expertise. To realize the high performance and power efficiency promised by reconfigurable accelerators, hardware developers must navigate complex tasks such as designing explicit pipelines and manage signal propagation delay. This typically involves using low-level Register-Transfer Level (RTL) languages [5], [6], or providing detailed design hints to High-Level Synthesis tools (HLS) [6], [7]. These requirements create a substantial hurdle of use.

To address the complexity challenges of FPGA development, processor overlays implement general-purpose processors [8]–[13] and GPUs [14]–[16] using the reconfigurable logic of FPGAs, removing the hardware design expertise requirement by asking the programmer to only write software for the overlay processor. Unfortunately, their performance is often lacking compared to custom kernels due to the resource overhead of general-purpose control logic such as instruction decode or deep pipeline management.

This paper presents Labidus, a RISC-V soft processor overlay designed to bridge the performance gap against manually optimized accelerators. Labidus employs an on-chip cluster of RISC-V soft processors organized into tiles. Each processor is augmented with a pool of potentially complex, fused operators, which are automatically composed for each application via static analysis. This approach is similar to LegUp [17], but Labidus takes it one step further by organizing the new operators in an asynchronous fashion using a large on-chip completion queue, as well as stream-semantic access to memory. Asynchronous semantics allows high utilization of even high-latency custom operators with minimal hardware overhead, especially since the completion queue is implemented with dense on-chip BRAM. To further improve utilization, each custom operator is shared across four cores in each tile, and the static analysis tool determines the efficient number of each shared operator to instantiate in hardware.

The additional functionality introduced by Labidus is incorporated into the RISC-V ISA with minimal change. To minimize hardware overhead, three registers are chosen to implement stream-semantic behavior. The software can issue burst memory requests coupled with automatically repeating operations to minimize instruction overhead as well as overlap long-latency custom instructions.

We evaluate Labidus on multiple scientific and signal processing workloads with kernels which are more complex compared to regularly structured matrix multiplication variants, against existing publications on best-effort optimized accelerators. Compared to optimized RTL and HLS implementations, Labidus demonstrates competitive end-to-end performance, sometimes even **outperforming expert-optimized implementations**. Labidus demonstrates especially high performance efficiency on applications with complex control, such as Cholesky decomposition. Compared to conventional soft processors and published soft GPU performance numbers, Labidus typically achieves an order of magnitude higher performance efficiency. It achieves over 50× the performance per chip resources compared to a softcore Microblaze, and almost 10× compared to state-of-the-art published soft GPUs.

In the trade-off between performance and productivity for accelerator development, Labidus achieves high performance

2160-052X/25/$31.00 ©2025 IEEE
DOI 10.1109/ASAP65064.2025.00018

efficiency based only on pure software programming. From the perspective of developers considering FPGA acceleration for an application, we believe Labidus can be one of the first and most convenient methods to attempt before potentially spending serious effort on optimized accelerator.

The rest of this paper is organized as follows: We present background and related works in Section II. We first introduce the programming model of Labidus in Section III, then present the accelerator and Labidus cluster architecture in Section IV. Evaluations against other published work are presented in Section V, and we conclude with discussions in Section VI.

II. Background and Related Work

A. Soft Processors and Overlays for FPGAs

One of the most critical hurdles of reconfigurable hardware acceleration is difficulty of programming. Tools such as High-Level Synthesis (HLS) tools try to overcome this by translating high-level programming languages such as C into hardware [6]. However, HLS tools can typically generate efficient hardware on very regular applications such as matrix multiplication, but require heavy hardware-aware optimization guidance from the programmer for irregular ones with dynamic loop bounds [6], [18].

As general-purpose processors are suitable for irregular applications with complex flow control, soft processors are also popular choices for implementing whole, or part of applications on an FPGA. The main benefit of soft processors is ease of programming thanks to established software programming languages and compilers. Many soft processor offerings are available including MicroBlaze, NIOS, RISC-V variants [19]–[22], as well as soft GPUs [23]. Some FPGA overlays deploy a pool of soft processors as the programming fabric for accelerator development [12].

Since soft processors are inevitably less efficient than ASIC processors, FPGA softcores often try to take advantage of the reconfigurable characteristic by adding application-specific modifications, such as application-specific coprocessors [8], or trimming unused components based on target software analysis [23]. Despite these efforts, soft processors are typically significantly slower compared to application-specific accelerators, as we will demonstrate.

B. Stream-Semantic Dataflow Architectures

Many architectural approaches are being explored to improve computation utilization of general-purpose processors by minimizing the control and data movement instruction overhead. Stream-semantic architectures decouple the memory access and execution units [24], [25], and include programmable memory access units which supplies the execution units with streams of data with low instruction overhead. Some such architectures implement execution units as a pool of dataflow operators [24], while some incorporate streams into existing ISAs [26], [27]. Some architectures use Stream-Semantic Registers (SSR), where the stream is mapped to an existing register file slot [25], [28].

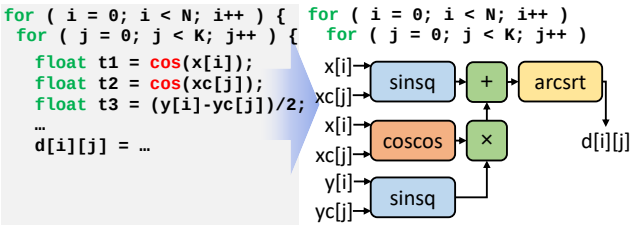


Fig. 1: Labidus-optimized Haversine distance.

Another promising architecture for efficient computation is queue machines, which use a FIFO queue as temporary memory instead of randomly addressable registers, have been explored due to their affinity to dataflow graphs [29], or ease of extracting parallelism [30], [31], demonstrating enough flexibility to efficiently express general computation. Queue machines are suitable for extracting parallelism by breadth-first traversing a dataflow graph [31]. With a large enough queue, a queue machine can extract large amounts of instruction-level parallelism without costly hardware components such as register renaming and reorder buffers [32]–[34]. There has also been success in algorithmically translating high-level languages to queue-machine programs [35].

Some FPGA implementations of queue machines have existed, but many suffer the resource overhead of implementing large queue memory using reconfigurable resources instead of dedicated BRAM [36].

III. Labidus Programming Model

Labidus targets FPGA accelerator cards like those used in Amazon AWS or Alibaba Cloud, with its own DRAM resources. Labidus separates software code into two regions: *Host* software, and *Kernel* software. The kernel software is translated into an accelerator by the Labidus compiler, and the host software calls the generated accelerator according to its internal logic. We emphasize that the programming environment described below applies to the *kernel* software, and the interaction between host and kernel software is left to the programmer, similar to existing HLS environments such as Vitis HLS.

A. Development Environment

Programming a Labidus accelerator is *entirely software-defined* from the programmer perspective, since it is a softcore overlay architecture. The programmer simply writes the code for the kernel software in pure C software, and points the Labidus static analysis tool to the compute-heavy portion

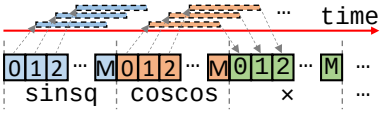


Fig. 2: Operator latency can be hidden by concurrently processing M inputs.

of the code. The static analysis tool generates two things: It generates HDL for the *Labidus hardware*, consisting of cores with application-specific optimizations such as the pool of operators for custom instructions. It also generates a modified version of the kernel software augmented with calls to Labidus internal functions, which express the same computation using asynchronous custom instructions for efficient execution. The HDL and software code is compiled with off-the-shelf HDL compilers (e.g., Vivado) and GCC, respectively. We note that the current Labidus library requires the programmer to explicitly specify the region of code to be offloaded to the operator pool. We plan to automate this in the future.

Figure 1 illustrates a case study of how the static analysis tool generates an optimized operator pool for user-defined software, using K-means with the Haversine distance metric. On the left side, the user provided software code for looping over N coordinates and calculating their distance to each of K means, using the complicated, but important, Haversine distance metric for the surface distance on a globe. Labidus analyzes the tree of operations in the loop body, and fuses operators starting from the leaf, as long as the fused operations still have two input operands and one output. In Figure 1, the tool discovered three fused operators: $sinsq(a,b) = sin^2(x-y)/2$, $coscos(a,b) = cos(a) \times cos(b)$, and $arcsrt(a) = 2R \times arcsin(sqrt(a))$, ($R$ is a constant) in addition to normal multiplication and addition. Due to the complexities of the trigonometric functions, each fused operator has dozens of cycles of latency. However they are also fully pipelined, and can ingest a new input every cycle.

The fused instructions are made available to the user software via an asynchronous interface made possible via an on-chip completion queue, which allows the soft processor to hide the operator latencies across multiple instructions. The size of the completion queue is an architectural parameter, which the Labidus compiler uses for efficient optimization. Given a completion queue of size M, the $N \times K$ loop iterations described in Figure 1 can be broken into blocks of M by the modified software at runtime. Inputs of each block can be processed concurrently, and the M eventual outputs are buffered in the completion queue, and then either re-used for future operators or streamed to the cache. As illustrated in Figure 2, each operator is run M times. The results of $sinsq$ in the completion queue are streamed to the cache while the input to $coscos$ is being streamed in. The $\times$ operator's two input operands come from memory ($sinsq$ results) and from the completion queue ($coscos$ results). As long as the first $coscos$ operation is completed in the queue before the first $\times$ operation is issued, the system will not stall, despite the long latency of the fused operation.

B. Accelerator Execution

From the point of view from host software, executing the accelerator generated by Labidus is very similar to most other popular accelerators, including GPUs. The FPGA is first reconfigured with the compiled bitfile of the HDL code, and then the compiled kernel software binary is loaded onto all cores of the resulting on-chip RISC-V cluster. Once both hardware and software is programmed, the host software simply copies the data to be processed over PCIe to FPGA DRAM, and then issues a start signal to all cores at once. Computation results are copied back to host, also over PCIe.

IV. Labidus System Architecture

The Labidus hardware is programmed onto an FPGA chip connected to a host server via an interconnect such as PCIe. Within the FPGA, a Labidus accelerator consists of multiple *Tiles* of RISC-V cores, where each tile consists of four cores. Figure 3 describes a Labidus accelerator with three tiles, with the internal architecture of one tile also shown. The on-chip network forms a hierarchy, where all cores within a tile form a ring, and all tiles, memory, and PCIe controllers form a 2D mesh network. We note that the number of cores per tile is a configurable parameter, although 4 is used as the default value.

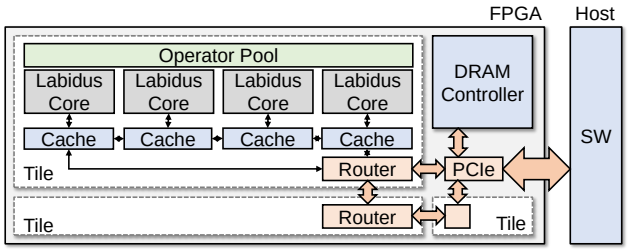


Fig. 3: A Labidus accelerator consists of a network of one or more tiles communicating with local DRAM and host.

A. Labidus Core Architecture

Labidus uses a simple RISC-V (RV32I) soft processor to drive the operator pool and memory I/O operations. Each core includes an asynchronous completion queue to hide the latencies of complex fused operators as well as memory operations. An example core is illustrated in Figure 4 with two operators in the pool.

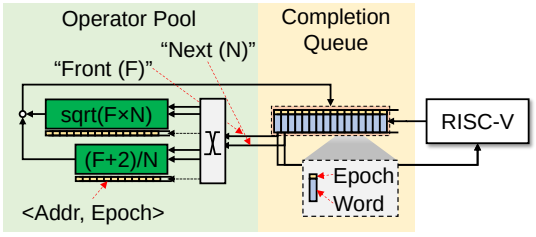


Fig. 4: A simple asynchronous completion queue machine hiding long operator latencies.

Labidus constructs an operator pool with a small number of complex operators by fusing many simple ones, in order to reduce the number of RISC-V instructions required to drive them. Such fused operators will typically have many cycles of latency, especially if they are floating-point operations.

Instead of using a deep pipeline to hide these latencies, Labidus takes advantage of the FPGA-specific characteristic

where embedded BRAM is denser than the programmable logic, to implement a large BRAM completion queue. On the one hand, a deep pipeline incurs large chip resource overhead in soft processors because it also needs a large register file either in the ISA or via register renaming, which in turn requires a large scoreboard to handle hazards. In contrast, a BRAM queue requires much less resources.

A large completion queue has two benefits: It can hide latencies of complex operators by supporting many concurrent operations in flight. It can also reorder the potentially out-of-order results from the operator pool caused by differences in operator latencies. Even long-latency operations will not cause pipeline bubbles as long as the results of each operation is entered into the queue before it reaches the head, and a large BRAM queue can support many slots for many concurrent operations, which in turn helps hide long operator latencies.

The queue is implemented as a circular buffer in a fixed-sized BRAM block. When an operation is issued, a slot is reserved at the tail pointer, and the reserved slot is asynchronously filled in (completed) whenever the results become available. Attempting to access a reserved but uncompleted slot will stall. We use an epoch scheme depicted in Figure 4, to keep track of completed slots in the buffer with low hardware overhead. A global epoch register is incremented whenever the tail pointer wraps around the memory. When requesting an operation, each request is tagged with the current epoch but the buffer slot is untouched. This tag follows the request and the eventual response which is updated to the completion queue. A slot has not been completed if its epoch is smaller than the previous one.

B. Stream-Semantic Memory System Architecture

Each Labidus core is equipped with a stream semantic cache for convenient and high-performance off-chip memory access. We opt to implement a cache in favor of programmer productivity. Even though an explicitly-managed scratchpad had roughly 10% less resource overhead, this comes at the cost of precious programmer productivity. The stream-semantic cache is not part of the RISC-V processor's memory hierarchy, and is exclusive to the completion queue. Figure 5 illustrates the difference between the stream-semantic cache and the RISC-V processor's local memory ("mem") where the program binary is loaded and the software load/store accesses. The completion queue, coupled with the stream-semantic cache, serves as the junction for all data movement between the core, off-chip memory, and the operator pool.

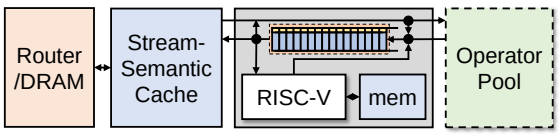


Fig. 5: Each Labidus core is equipped with a stream-aware cache capable of supplying bursts of data I/O.

The cache is stream-semantic in the sense that memory I/O to the cache system is made in units of bursts, instead of words. The RISC-V software is responsible for requesting bursts of memory I/O, and stream semantics reduces the number of RISC-V processor cycles spent generating memory requests. Requests are made through additional I and S type instructions, with modified encodings to support burst-related parameters. The 12 bits of immediate encoding space provided for the typical I and S type instructions are split into 6 bits of *burst length*, and 5 bits of *repeat count*, and one bit *uncached* parameters. The repeat count parameter specifies how many times the same memory region should be accessed repeatedly, which is very useful when small data structures (such as coefficients) must be read repeatedly. The uncached flag prevents the memory request from being cached if the request causes off-chip memory access.

Since Labidus implements stream-semantic I/O, bursts of memory reads can be read one after another by reading from a single register, and bursts of memory writes can be issued by writing repeatedly to a single register. Since augmenting all registers for stream semantics is expensive, Labidus restricts stream semantics to three registers: `x29`, `x30` and `x31`.

Each cache includes a single BRAM module for the cache, as well as a bypass queue in BRAM for I/O requests with the uncached flag set. We discovered that for single-function accelerators like the applications we explore in Section V, a simple direct-mapped cache offers competitive performance at a low resource overhead. This is also thanks to the completion queue, which lowers the pressure on the memory system.

The memory controller also supports a memory-mapped *barrier* command for inter-core synchronization, which stops the memory controller from processing any more commands until the previously issued ones are completed.

C. Operator Pool and Trimming

The Labidus operator pool is designed to improve the utilization of each operator instance, by supporting automatically repeating instructions on a stream of data generated by the stream semantic cache, as well as trimming underutilized operator instances. All hardware instances for operators are organized into a 3D mesh architecture where the same operator are grouped into the same Z-level. Operator requests must be multi-hop routed to the intended operator over multiple cycles, but this does not cause performance issues thanks to the completion queue.

Operators in the pool are invoked via a set of instructions added via a new R-type instruction opcode. The three source and target registers are encoded as per normal as part of the R-type instruction encoding, as well as the operation to invoke, which is encoded using the func7 field as usual. Additionally, the func3 field is used to encode the *repeat*, parameter, which encodes the number of times the operation should be invoked.

The repeat field is necessary for performance, because having the RISC-V software issue operator commands means that operator commands cannot be issued while the RISC-V core is executing other instructions such as memory I/O. This can cause the operator pool to starve, always waiting for the processor to invoke it. To overcome this limitation,

the operator pool ISA includes the `repeat` field which can be configured to repeat instructions, similar to the Floating-Point Repetition (FREP) extension proposed for RISC-V [25]. By issuing a single command which causes repeated operator execution, it helps overlapping the operator pool execution and other control and memory access instructions. We note that the repeat parameter is only useful with the stream-semantic registers, because otherwise it will simply execute the same computation multiple times with the same register value. Only when coupled with the stream-semantic registers and burst memory operations (implemented with `x29`, `x30` and `x31`) are these operations at their full potential.

We note that Labidus can support much more than the 64 instruction types the 6 bit opcode can express. The Labidus hardware template can have hundreds or instructions, and new, custom instructions can be added very easily. The 6-bit opcodes are assigned to each instruction during hardware generation, only to the operators that are actually used for this particular design. As a result, each hardware instance can support up to 64 different operators from a pool of hundreds.

Even with the help of the repeat parameters for both operations and memory I/O, not all operators can be kept busy at all times. To overcome the resource waste, Labidus pools and custom operation units per tile into a shared operator pool. The static analysis tool conservatively estimates the relative frequency of operations to *trim* some instances. Based on popularity, each operator can be instantiated anything between one to four times for each tile, improving utilization of each remaining operator and saving resources. Since execution units are instantiated only for operators that are actually used, typically we end up with operator pools with a few units.

V. Performance Evaluation

A. Implementation Details

Labidus is evaluated on the Xilinx Virtex Ultrascale chip using the Amazon AWS F1 environment, and was synthesized at 250 MHz. The LUT utilization varies according to the operator pool configuration, and spans between 3.4% (FFT) to 4.2% (Haversine). The resource utilization of the rest of a tile, including four RV32I cores, four stream-semantic caches, and four completion queues, is fairly consistent. The cache and processor also consume 160 BRAM blocks (7%) in the current configuration. Beyond the tile itself, both the PCIe interface and DRAM controller on the VC707 platform consumes about 4% of the LUT resources.

1) Operator pool trimming: Because each RISC-V soft processor and the completion queue interface typically supports multiple custom operations, the number of each operator instance required per 4-core tile is typically much smaller than four. Table I shows the configuration of the operator pool as determined by the static analysis tool. Some operators are application-specific, (e.g., $sqrt$ for Cholesky, $sinsq$ and other fused operators for KMeans), meaning including all operators for all cores will result in significant resource overhead. Even only including the actually used operators, but 4 per tile, results in a significantly larger overhead, as seen in the left chart of Figure 9.

TABLE I: Number of operators instantiated per 4 Labidus cores.

	$\times$	$\div$	+	$\sqrt{x}$	$sinsq$	$coscos$	arcsrt
Cholesky	2	1	2	1	-	-	-
FIR	2	-	2	-	-	-	-
FFT	2	-	3	-	-	-	-
KMeans	2	-	2	-	1	1	1

The static analysis tool trims operators very conservatively, and as a result trimmed configurations did not result in any performance loss. The average utilization of each instantiated operator is shows on the right side of Figure 9, and we see that utilization improved by over 100% points on average across all benchmarks.

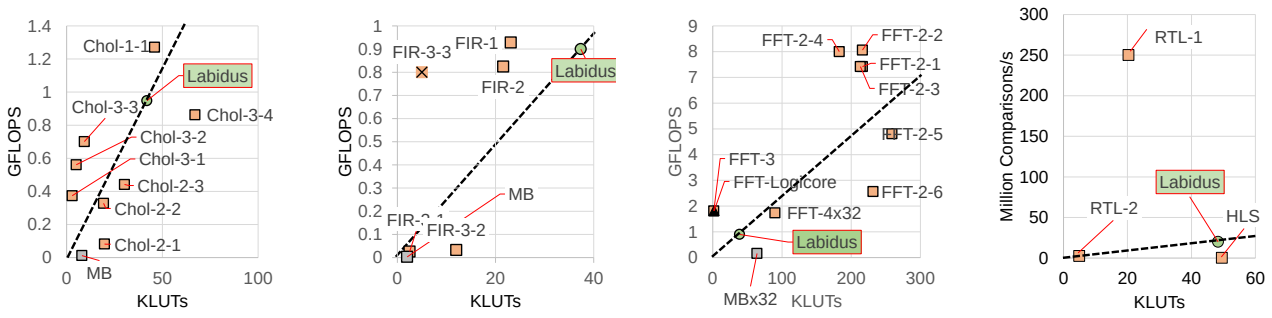
2) Resource efficiency of the completion queue: For comparison, we evaluated a simple modification to the RISC-V processor which adds a 1024-slot reorder buffer (but without corresponding ISA modifications) to estimate the resource utilization of a processor with similar latency-hiding capability. Such deep pipeline design required over 80K LUTs (˜7%). On the other hand, a baseline resource utilization of an tile with 1024-size completion queue, excluding the operator pool, consumes about 38K LUTs (˜3%), over 2$\times$ improvement in resource efficiency.

B. Application Performance Evaluation

In this section, we evaluate Labidus accelerator implementations against RTL and HLS designs, both of which **require hardware-specific expertise** for optimization. Even though Labidus requires *zero* hardware design expertise, we show that not only does it achieve competitive performance, it sometimes even outperforms manually optimized RTL and HLS.

1) Comparing Against Published Designs: To make sure we do not use under-optimized HLS and RTL designs, we evaluate Labidus against a suite of published, expert-optimized implementations. These published designs are results of many person-days of optimization by experts. However, published accelerators are often implemented on different platforms (e.g., Xilinx vs. Altera, different clock speeds, etc). To err on the side of caution, we normalize performance numbers with the following rules: The performance of accelerators with slower clock speeds are scaled proportionally to 250 MHz. For example, an accelerator reported at 10 GFLOPS at 100 MHz is evaluated as 25 GFLOPS. Different FPGA families are normalized using total LUT inputs. For example, if a published system used 600 of Altera Stratix II 4-input ALUTs, we evaluate it as only using 400 Xilinx Ultrascale 6-input LUTs. While this approach is by no means perfect, we believe it is fairer than using our own designs potentially lacking application-domain expertise.

2) Applications: Applications were chosen based on importance, and availability of recent FPGA accelerator publications. We present applications of similar complexity with comparable architectural research publications [24]. In the



(a) Cholesky decomposition. (b) Finite Impulse Response. (c) Fast Fourier Transform. (d) K-Means with Haversine.

Fig. 6: Performance efficiency comparisons of representative applications.

interest of space, we omit some benchmarked applications including parallel reduction, LU decomposition, DBSCAN, and some signal processing algorithms which show similar characteristics as the ones already represented.

Cholesky Decomposition: Cholesky decomposition is one of the fundamental HPC applications. A large matrix is stored in off-chip DRAM, and each core works on a stream of sub-matrices of size 256. We assume the whole matrix fits in distributed on-chip memory, to remove the dependency on DRAM performance. Compared systems include OmpSs (Chol-1 [37], 2018), RTL (Chol-2 [38], 2013), HLS (Chol-3 [18], 2018) and Microblaze softcore (MB).

Finite Impulse Response Filter: We implement a simple single-core implementation of FIR, and assign one independent stream per each core. Compared systems include the Xilinx reference implementation (FIR-1, 2017), Technical report on HLS from ANL (FIR-2 [39], 2017), Soft GPU (FIR-3 [40], 2017) and Microblaze (MB).

Fast Fourier Transform: The Labidus software simply parallelizes 256-point FFTs across available cores. Compared systems include the Xilinx reference (FFT-1, 2012), OpenCL (FFT-2 [41], 2013), HLS (FFT-3 [42], 2017), RTL (FFT-4x32 [43], 2016), and 32 parallel Microblaze cores (MBx32). The resource and performance of the RTL system (FFT-4) is uniformly multiplied 32 times for visibility.

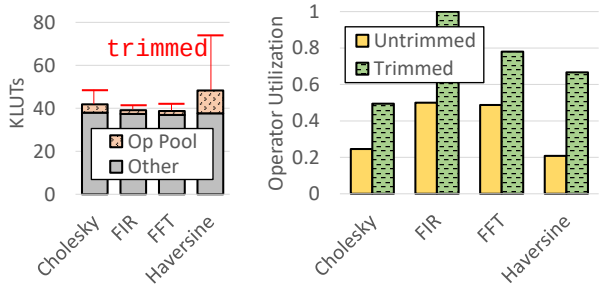


Fig. 7: Trimming the operator pool results in lower LUT usage, and better per-operator utilization without harming performance.

K-means Clustering with Haversine Distance: We implement the distance comparison and cluster assignment distributed over multiple cores, using the haversine distance metric for geolocation data. We compare against our internal RTL (RTL-1) and HLS (HLS) implementations, as well as a published RTL one (RTL-2 [44], 2021).

Omitted: Matrix Multiplication, Deep Neural Networks: We emphasize that target applications of Labidus are new exploratory ones requiring rapid prototyping, especially those with complex control logic which make prototyping difficult. As a result, we intentionally omit regularly-structured, dense algorithms such as matrix multiplication and deep neural networks. Provided for completeness, on matrix multiplication and N-body physics simulation, published RTL cores performed on average 4 to 7 times faster than Labidus given similar resources, thanks to low control overhead. While Labidus still provides superior throughput compared to CPUs, we note that such popular kernels with simple control is not the target of Labidus.

3) Results: Results are based on a single tile unless otherwise stated. Each tile has been automatically configured and pruned by the static analysis tool.

Figure 6 compares the performance per LUT usage of each implementation. The dotted line depicts the performance efficiency of Labidus. Systems above this line can achieve higher performance compared to Labidus, given the same chip resources. Considering that implementations have varying resource optimization targets, this is a good way to estimate control overhead relative to actual DSP throughput, and project how much performance we can get from the given chip resources.

Figure 6 shows that Labidus achieves competitive performance efficiency compared even to well-optimized designs, even outperforming RTL code when computation patterns are irregular (e.g., Chol-2). It also shows that conventional soft processors (e.g., Microblaze, FIR-3) are not very performance efficient. We note that FIR-3-3 reports very high performance thanks to the static optimization of fixing the window size in hardware, so we report it for completeness but marked with an

X. We also observe high variability of performance with high-level programming approaches (e.g., FFT-2 with OpenCL, vs. Chol-3 with HLS).

C. Comparison Against Soft Processors

To emphasize the benefits of the architectural optimization of Labidus, we also compare the performance and resource efficiency of Labidus against published soft processor designs, including the Xilinx Microblaze soft CPU and various published soft GPUs. The compared systems are: Xilinx Microblaze [45], Kamalelidin et.al. (2020) [22], FlexGrip (2013) [15], FGPU (2016) [16], and DO-GPU (2021) [14].

Performance translation across FPGA products and clock speeds were done in the same manner. We compare against DO-GPU without the Macro optimization, which requires writing custom hardware kernels. Some systems without concrete performance numbers were omitted. We note Kamalelidin et.al. compares against many works including GRVI Phalanx [20] and demonstrate superior performance efficiency.

1) Configurations: We evaluate these systems using conditions most favorable to them. For example, we use the best measured MFLOP performance of Microblaze (6 MFLOPS) when performance is given as multiples of microblaze, even if it does not say an FPU was used. If multiple configurations are presented, we only use most efficient one. We normalize across FPGA vendors and models using total number of LUT inputs, as well as normalize clock speed to 250 MHz when comparison systems are slower. DO-GPU achieves different clock speeds depending on the target application, so we present the best-case 250 MHz implementation.

For Labidus, we assume a 80% utilization across all operators with a single, 4-core tile. None of our evaluated applications showed operator utilization below 80%. To err on the side of fairness, the tile was configured with full operator replication with no trimming.

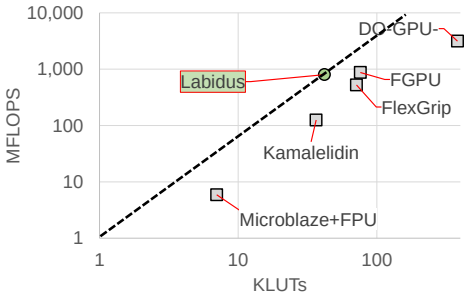


Fig. 8: LUT utilization and performance of soft processor implementations. Points right of the dotted line are less efficient than Labidus.

2) Results: Figure 8 illustrates the performance efficiency of soft processor designs using the Cholesky decomposition example, where Labidus demonstrates superior resource and performance efficiency.

D. Operator Pool Utilization

Thanks to the configurable architecture of the shared operator pool, Labidus can achieve higher utilization of each operator instances by trimming underutilized ones. We have already presented the trimmed configuration of operators for the six target benchmarks in Table I. Figure 9 shows the increase in utilization for each operator after trimming. Utilization of most operators have increased significantly, even rarely-used ones like SqrtCube (`SC`) , which is only used by the N-Body example. Some operators still have very low utilization, such as division for Cholesky and LU decomposition. These may be offloaded to RISC-V software to save chip resources, but the complexity of a floating-point division using the RV32I ISA caused enough performance degradation for us to select hardware operator instances.

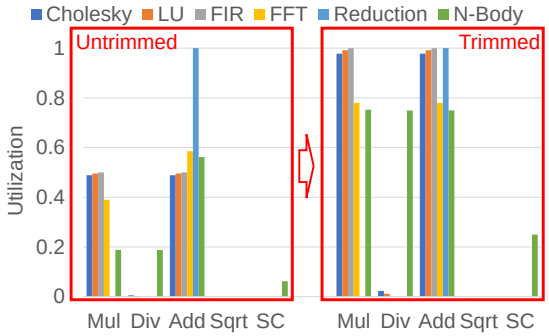


Fig. 9: Trimming the operator pool results in better utilization without harming performance.

VI. Conclusion and Discussion

We have presented the design, implementation, and evaluation of Labidus, a configurable, parallel processor overlay for more productive hardware acceleration. Labidus augments a cluster of RISC-V soft processors with a trimmable pool of shared operators, an asynchronous completion queue to hide operator latencies, as well as stream-semantic memory operations which improve operator utilization and reduce chip resource overhead. We are working on further improving the programmability of Labidus, by automatically extracting and implementing custom kernels from legacy software [46].

Acknowledgements

This research is partially supported by the SRC JUMP 2.0 PRISM Center (000705769) and Samsung Electronics.

References

[1] M. Qasaimeh, K. Denolf, J. Lo, K. Vissers, J. Zambreno, and P. H. Jones, "Comparing energy efficiency of cpu, gpu and fpga implementations for vision kernels," in *2019 IEEE international conference on embedded software and systems (ICESS)*. IEEE, 2019, pp. 1–8.

[2] J. Fang, Y. T. Mulder, J. Hidders, J. Lee, and H. P. Hofstee, "In-memory database acceleration on fpgas: a survey," *The VLDB Journal*, 2020.

[3] T. Hamada, K. Benkrid, K. Nitadori, and M. Taiji, "A comparative study on asic, fpgas, gpus and general purpose processors in the o (nˆ 2) gravitational n-body simulation," in *2009 NASA/ESA Conference on Adaptive Hardware and Systems*. IEEE, 2009, pp. 447–452.

[4] E. Chung, J. Fowers, K. Ovtcharov, M. Papamichael, A. Caulfield, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman *et al.*, "Serving dnns in real time at datacenter scale with project brainwave," *iEEE Micro*, vol. 38, no. 2, pp. 8–20, 2018.

[5] J. Cong, Z. Fang, Y. Hao, P. Wei, C. H. Yu, C. Zhang, and P. Zhou, "Best-effort fpga programming: A few steps can go a long way," *arXiv preprint arXiv:1807.01340*, 2018.

[6] R. Nane, V.-M. Sima, C. Pilato, J. Choi, B. Fort, A. Canis, Y. T. Chen, H. Hsiao, S. Brown, F. Ferrandi *et al.*, "A survey and evaluation of fpga high-level synthesis tools," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 10, 2015.

[7] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, "High-level synthesis for fpgas: From prototyping to deployment," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 4, pp. 473–491, 2011.

[8] J. Kadlec, R. Bartosinski, and M. Danek, "Accelerating microblaze floating point operations," in *2007 International Conference on Field Programmable Logic and Applications*. IEEE, 2007, pp. 621–624.

[9] D. Capalija and T. Abdelrahman, "A coarse-grain fpga overlay for executing data flow graphs," in *The Second Workshop on the Intersections of Computer Architecture and Reconfigurable Logic*. Citeseer, 2012.

[10] Y. Yu, C. Wu, T. Zhao, K. Wang, and L. He, "Opu: An fpga-based overlay processor for convolutional neural networks," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 1, 2019.

[11] Y. Yu, T. Zhao, K. Wang, and L. He, "Light-opu: An fpga-based overlay processor for lightweight convolutional neural networks," in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2020, pp. 122–132.

[12] R. Rashid, J. G. Steffan, and V. Betz, "Comparing performance, productivity and scalability of the tilt overlay processor to opencl hls," in *International Conference on Field-Programmable Technology (FPT)*. IEEE, 2014.

[13] C. E. LaForest, *High-speed soft-processor architecture for FPGA overlays*. University of Toronto (Canada), 2015.

[14] R. Ma, J.-C. Hsu, T. Tan, E. Nurvitadhi, R. Vivekanandham, A. Dasu, M. Langhammer, and D. Chiou, "Do-gpu: Domain optimizable soft gpus," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2021, pp. 140–144.

[15] K. Andryc, M. Merchant, and R. Tessier, "Flexgrip: A soft gpgpu for fpgas," in *2013 International Conference on Field-Programmable Technology (FPT)*. IEEE, 2013, pp. 230–237.

[16] M. Al Kadi, B. Janssen, and M. Huebner, "Fgpu: An simt-architecture for fpgas," in *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2016, pp. 254–263.

[17] A. Canis, J. Choi, M. Aldham, V. Zhang, A. Kammoona, J. H. Anderson, S. Brown, and T. Czajkowski, "Legup: high-level synthesis for fpga-based processor/accelerator systems," in *ACM/SIGDA international symposium on Field programmable gate arrays*, 2011, pp. 33–36.

[18] Y.-k. Choi and J. Cong, "Hls-based optimization and design space exploration for applications with variable loop bounds," in *2018 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 2018.

[19] S. Mashimo, A. Fujita, R. Matsuo, S. Akaki, A. Fukuda, T. Koizumi, J. Kadomoto, H. Irie, M. Goshima, K. Inoue *et al.*, "An open source fpga-optimized out-of-order risc-v soft processor," in *2019 International Conference on Field-Programmable Technology (ICFPT)*. IEEE, 2019.

[20] J. Gray, "Grvi phalanx: A massively parallel risc-v fpga accelerator accelerator," in *International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2016, pp. 17–20.

[21] M. Naylor, S. W. Moore, and D. Thomas, "Tinsel: a manythread overlay for fpga clusters," in *International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2019.

[22] A. Kamaleldin, S. Hesham, and D. Göhringer, "Towards a modular risc-v based many-core architecture for fpga accelerators," *IEEE Access*, vol. 8, pp. 148 812–148 826, 2020.

[23] P. Duarte, P. Tomas, and G. Falcao, "Scratch: An end-to-end application-aware soft-gpgpu architecture and trimming tool," in *Annual IEEE/ACM International Symposium on Microarchitecture*, 2017.

[24] T. Nowatzki, V. Gangadhar, N. Ardalani, and K. Sankaralingam, "Stream-dataflow acceleration," in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2017.

[25] F. Zaruba, F. Schuiki, and L. Benini, "Manticore: A 4096-core risc-v chiplet architecture for ultraefficient floating-point computing," *IEEE Micro*, vol. 41, no. 2, pp. 36–42, 2020.

[26] Z. Wang and T. Nowatzki, "Stream-based memory access specialization for general purpose processors," in *International Symposium on Computer Architecture (ISCA)*. IEEE, 2019, pp. 736–749.

[27] Z. Wang, J. Weng, J. Lowe-Power, J. Gaur, and T. Nowatzki, "Stream floating: Enabling proactive and decentralized cache optimizations," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 640–653.

[28] F. Schuiki, F. Zaruba, T. Hoefler, and L. Benini, "Stream semantic registers: A lightweight risc-v isa extension achieving full compute utilization in single-issue cores," *IEEE Transactions on Computers*, vol. 70, no. 2, pp. 212–227, 2020.

[29] H. Schmit, B. Levine, and B. Ylvisaker, "Queue machines: hardware compilation in hardware," in *IEEE Symposium on Field-Programmable Custom Computing Machines*. IEEE, 2002, pp. 152–160.

[30] B. A. Abderazek, T. Yoshinaga, and M. Sowa, "High-level modeling and fpga prototyping of produced order parallel queue processor core," *The Journal of Supercomputing*, vol. 38, no. 1, pp. 3–15, 2006.

[31] S. Okamoto, H. Suzuki, A. Maeda, and M. Sowa, "Design of a superscalar processor based on queue machine computation model," in *1999 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM 1999). Conference Proceedings (Cat. No. 99CH36368)*. IEEE, 1999.

[32] H. Hoshino, B. A. Abderazek, and K. Kuroda, "Advanced optimization and design issues of a 32-bit embedded processor based on produced order queue computation model," in *IEEE/IFIP International Conference on Embedded and Ubiquitous Computing*, vol. 1. IEEE, 2008.

[33] M. Sowa, B. A. Abderazek, and T. Yoshinaga, "Parallel queue processor architecture based on produced order computation model," *The Journal of Supercomputing*, vol. 32, no. 3, pp. 217–229, 2005.

[34] M. Sowa, B. A. Abderazek, S. Shigeta, K. Nikolova, and T. Yoshinaga, "Proposal and design of a parallel queue processor architecture (pqp)." in *IASTED PDCS*, 2002, pp. 549–554.

[35] A. Canedo, B. A. Abderazek, and M. Sowa, "A gcc-based compiler for the queue register processor (qrp-gcc)," in *International Workshop on Modern Science and Technology*, 2006, pp. 250–255.

[36] A. B. Abdallah, "Soft-core processor for low-power embedded multicore socs," in *Multicore Systems On-Chip: Practical Software/Hardware Design*. Springer, 2013, pp. 195–213.

[37] J. Bosch, X. Tan, A. Filgueras, M. Vidal, M. Mateu, D. Jiménez-González, C. Álvarez, X. Martorell, E. Ayguadé, and J. Labarta, "Application acceleration on fpgas with ompss@ fpga," in *International Conference on Field-Programmable Technology (FPT)*. IEEE, 2018.

[38] J. Luo, Q. Huang, S. Chang, X. Song, and Y. Shang, "High throughput cholesky decomposition based on fpga," in *International Congress on Image and Signal Processing (CISP)*, vol. 3. IEEE, 2013.

[39] Z. Jin, H. Finkel, K. Yoshii, and F. Cappello, "Evaluation of the fir example using xilinx vivado high-level synthesis compiler," Argonne National Lab.(ANL), Argonne, IL (United States), Tech. Rep., 2017.

[40] M. Al Kadi, B. Janssen, and M. Huebner, "Floating-point arithmetic using gpgpu on fpgas," in *2017 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2017, pp. 134–139.

[41] J. Andrade, V. Silva, and G. Falcao, "From opencl to gates: The fft," in *2013 IEEE Global Conference on Signal and Information Processing*. IEEE, 2013, pp. 1238–1241.

[42] G. Xu, T. M. Low, J. Hoe, and F. Franchetti, "Optimizing fft resource efficiency on fpga using high-level synthesis," *Proc. IEEE HPEC*, 2017.

[43] M. Ibrahim and O. Khan, "Performance analysis of fast fourier transform on field programmable gate arrays and graphic cards," in *2016 International Conference on Computing, Electronic and Electrical Engineering (ICE Cube)*. IEEE, 2016, pp. 158–162.

[44] B. Işik, "Area optimized fpga implementation of slant range calculation using haversine formula," in *2021 29th Signal Processing and Communications Applications Conference (SIU)*. IEEE, 2021, pp. 1–4.

[45] Xilinx, "Microblaze soft processor core," https://www.xilinx.com/products/design-tools/microblaze.html, Accessed November 2021.

[46] E. Aliaj, A. Krone-Martins, J. Garcia, and S.-W. Jun, "Farslayer: Turnkey acceleration of legacy software on commodity fpga cards," in *International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE, 2023.